Student: Ling Ling He

Submission Date: 19/12/2025

Mobile Robot Assembly and the Navigation Algorithm in a Real Environment

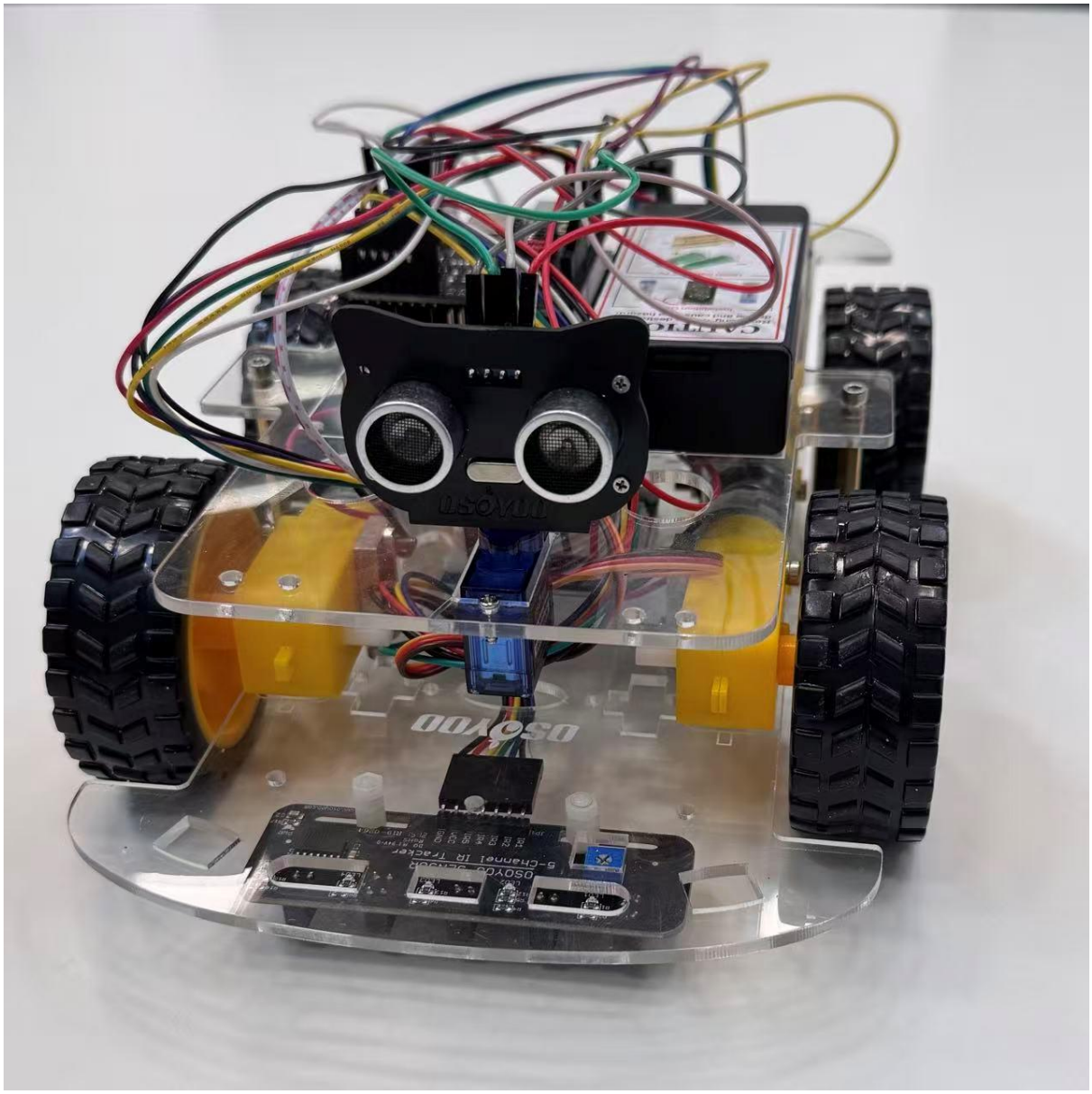


Figure 1

Abstract

The report presents the assembly, interfacing, and experimental evaluation of a four-wheeled autonomous mobile robot using the OSOYOO V2.1 Robot Car Kit. The laboratory investigated the performance of navigation algorithms, including line tracking, obstacle avoidance, and wall following in an actual environment. An Arduino UNO R3 microcontroller was used to interface sensors and actuators. Infrared line sensors enabled line-following behaviour, while an ultrasonic sensor was used for obstacle detection and distance measurement. The experimental results showed high accuracy within a short-range distance (10-40cm), with a maximum observed error of ±1 cm. The robot successfully navigated a cluttered environment using reactive obstacle avoidance with a return-to-line strategy. Limitations were identified, and potential improvements for navigation.

Table of Contents

1. Originality Declaration

I hereby certify that I understand the nature of plagiarism and that I understand the University's regulations regarding examination misconduct. I verify that I am the sole author of this report, except where explicitly stated otherwise.

2. Introduction

The purpose of this laboratory is to assemble a mobile robot, interface its sensors and actuators, and develop and test navigation algorithms such as line tracking, obstacle avoidance and wall-following in a real-world environment. Evaluating the performance of the ultrasonic sensor in sensing the distance to an obstacle.

The OSOYOO V2.1 Robot Car Kit is an Arduino-based, four-wheel differential drive mobile robot capable of moving via independent motor control. An Arduino microcontroller enables sensor data processing and motor actuation to implement navigation behaviours. The robot is equipped with a five-channel infrared(IR) tracking sensor for line following and an ultrasonic sensor for obstacle detection. The following sections will present the hardware configuration, algorithm implementation, experimental methodology and results analysis.

3. Robot Platform and Assembly

3.1. The OSOYOO V2.1 Robot Car

The OSOYOO V2.1 robot car is an Arduino-based kit for building and programming a robotic car. It employs a four-wheel differential drive configuration in which the left and right wheel pairs are driven independently. This allows the robot to rotate and move in a straight line.

3.2. Assembly Procedure

1. **Motion Assembly**: Four gear motors and an OSOYOO Model X motor driver were mounted on the lower car chassis. They are connected on the K1, K2, K3 and K4 sockets on the Model-X motor driver board.
2. **Power System**: A voltage metre was mounted at the back of the car on the lower car chassis, which was connected to the driver motor (GND-GND, VCC-12, VT-V0). It measured the voltage of the robot's power supply. A battery holder containing two 18650 lithium batteries was installed on the upper car chassis, which was to provide power to the system.
3. **Controller and Shield Installation**: The Arduino UNO R3 board and OSOYOO UART WIFI shield were mounted on the upper car chassis.
4. **Sensor Installation**: The ultrasonic sensor was mounted at the front of the robot with a servo sensor at the top of the upper car chassis. They worked together, rotating and detecting the position. The infrared line sensor was installed underneath the lower car chassis for line detection. The speed sensors were secured between the two front wheels to measure the speed, distance and control movement.
5. After assembly, the upper and lower chassis plates were fixed together.

4. Interfacing

4.1. Microcontroller: Arduino UNO R3 Board

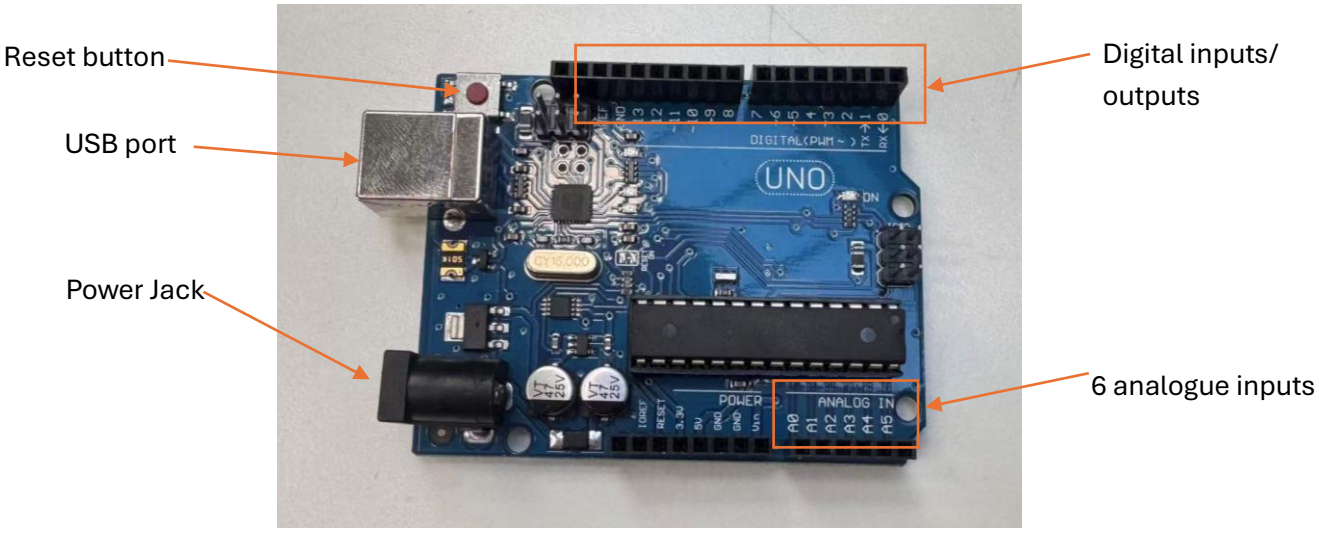


Figure 2

A microcontroller is a self-contained embedded computing system that enables real-time sensing, decision-making and actuation in a robotic system. In the laboratory, the Arduino UNO R3 microcontroller served as the central processing unit of the mobile robot. It has 14 digital input/output pins and 6 analogue inputs, which were sufficient for interfacing the sensors and actuations. It continuously acquired data from the infrared tracking sensor and the ultrasonic sensor, processed this information using navigation algorithms, and generated control signals for the gear motors and the servo motor. The board was programmed using Arduino IDE via a USB connection. Executed the autonomous navigation algorithm when powered by the onboard battery system.

4.2. Arduino Hardware Interface and Pin Configuration

The gear motors were connected to the motor driver, with direction controlled by digital pins and motor speed controlled using pulse-width modulation (PWM) outputs.

The five-channel infrared tracking sensor was connected to the analogue input pins A0-A4 of the microcontroller. Each sensor channel provides a digital signal indicating the presence or absence of the black line beneath the robot.

The motor driver was connected to the Arduino using both digital pins and PWM speed control pins. The right motor speed was controlled using PWM pin D3, and the left motor speed was controlled using PWM pin D6. Motor direction was controlled using digital pins D11 & D12 for the right motor and D7 & D8 for the left motor.

The ultrasonic sensor was interfaced using two digital pins. The trigger pin connected to Arduino digital pin D10, and the echo pin connected to D2. The microcontroller generates ultrasonic pulses from the trigger pin and measures the echo return time to calculate the distance to the obstacle.
The servo motor was connected to digital pin D9 to support PWM-based position control. It was used to rotate the ultrasonic sensor to scan the environment.

4.3. Arduino Software Interface

The Arduino Integrated Development Environment (IDE) was used to wire, compile and upload programs to the Arduino microcontroller. The code was first verified to check its correctness, and then uploaded to the physical robot via a USB connection. During execution, the Serial Monitor was used to visualise sensor readings, which helped to debug the navigation algorithms during testing.

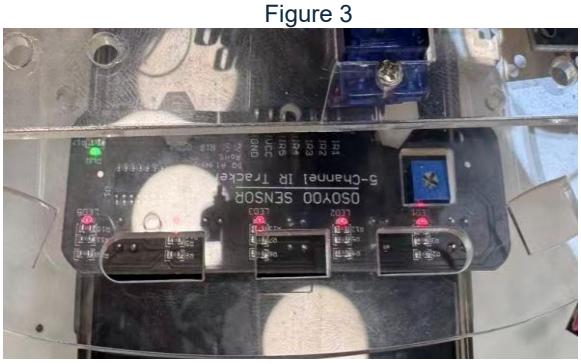
In addition to physical testing, the Tinkercad simulation environment was used to virtually model and test the ultrasonic distance measurement algorithm. It presented the sensor behaviour in a controlled virtual environment, which was a useful comparison with experimental results from a physical robot.

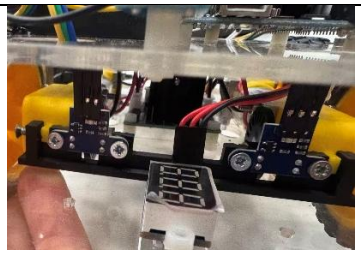
Conducting both physical experiments and virtual experiments increased the reliability of the algorithm and helped identify potential issues with the robot.

5. Sensors and environment

5.1. Sensors

The table below shows sensors used in the laboratory:

Sensor	Image	Function
Five-channel Infrared tracking Sensor	 Figure 3  Figure 4	Help the mobile robot detect and follow a black line on a light surface in the lab. Each channel detects a line by measuring the reflection of emitted infrared light from the surface for real-time automated line tracking. Red light indicates black detected. FIGURE 3 shows inactive IR LEDs when they are on a light surface. FIGURE 4 shows active 5 IR LEDs when they detect black.
Ultrasonic Sensor	 Figure 5	Detects objects, distance, and prevents collisions.

Servo Sensor	 Figure 6	 Figure 7	An encoder that rotates the ultrasonic sensor for scanning.
Speed Sensor	 Figure 8	 Figure 9	Measure wheel speed and distance.

5.2. Experimental Environment

Experiments were conducted on a flat laboratory table with thin black tape placed on a light surface and on the laboratory floor. A black box was positioned in front of the robot to test obstacle detection and avoidance (Figure 10-12).

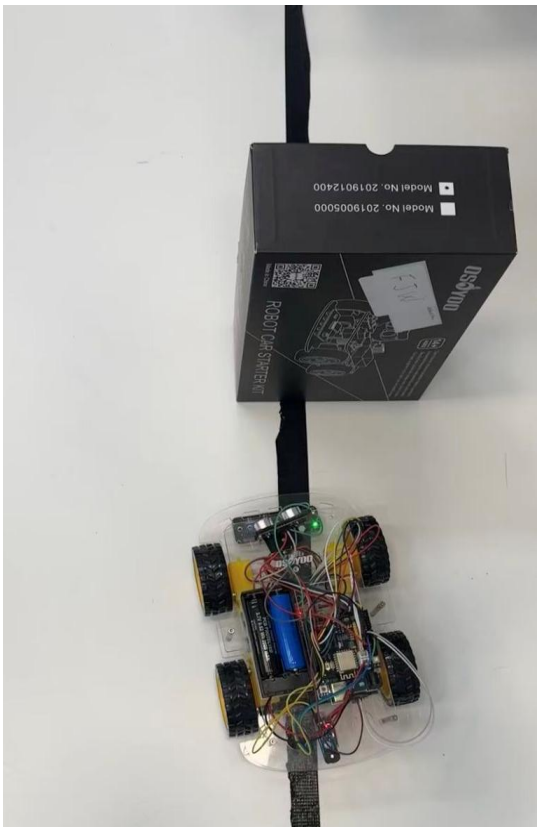


Figure 10

Figure 11

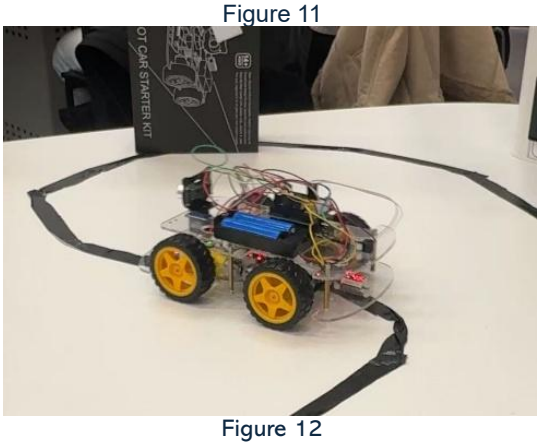


Figure 12

6. Navigation Algorithm

6.1. Line Tracking

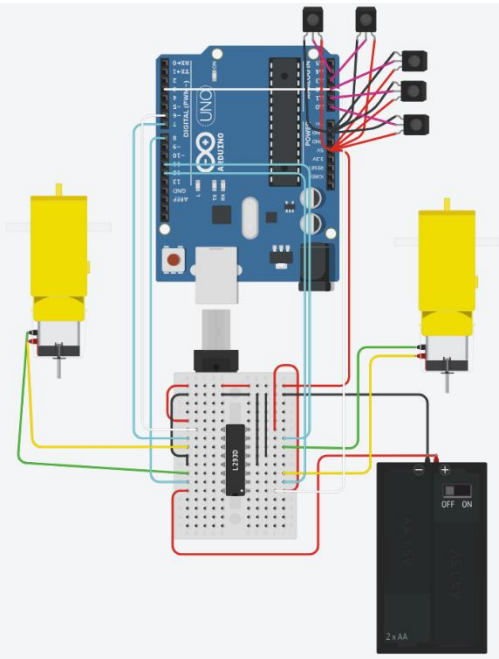


Figure 13

```
void loop() {
  String val = read_sensor_values();
  Serial.print("SENSORS: ");
  Serial.print(val);
  Serial.print(" → ");

  if (val == "10000"){
    go_Left();
    set_Motorspeed(FAST_SPEED, FAST_SPEED);
  }else if (val == "00001"){
    go_Right();
    set_Motorspeed(FAST_SPEED, FAST_SPEED);
  }else if (val == "01000" || val == "01100" || val == "00100"){
    go_Forward();
    set_Motorspeed(0, FAST_SPEED);
  }else if (val == "00010" || val == "00011" || val == "00110"){
    go_Forward();
    set_Motorspeed(FAST_SPEED, 0);
  }else{
    go_Forward();
    set_Motorspeed(MID_SPEED, MID_SPEED);
  }
}
```

Figure 14

6.2. Obstacle Avoidance Algorithm Flowchart

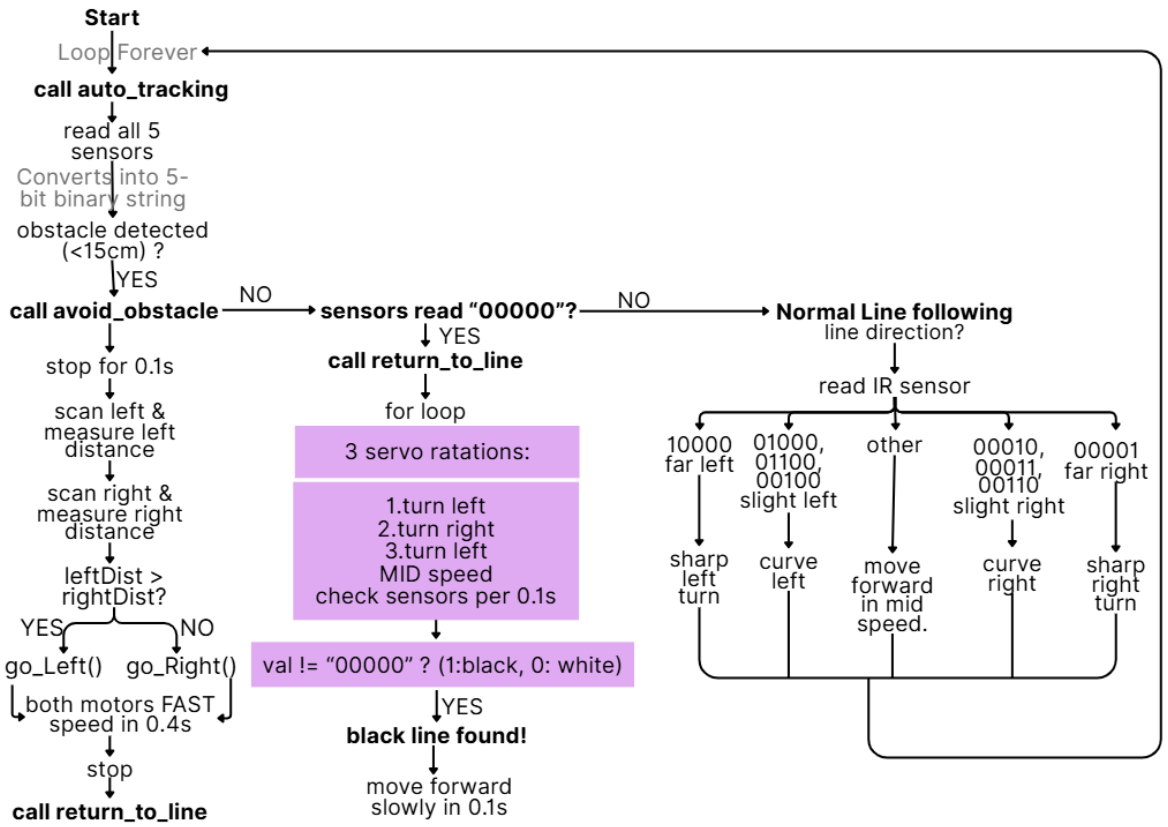


Figure 15

The line tracking used a 5-channel infrared line sensor that consists of 5 LEDs at the front of the robot. Each channel detects reflected infrared light to determine whether it is positioned over a black line. Sensor readings are processed as a binary pattern

to determine the robot's relative position to the line. Based on this pattern, the robot consistently adjusted motor speeds to steer left, right, or continue forward.

Obstacle avoidance is implemented using an ultrasonic sensor. When the measured distance falls closer than a predicted threshold (5cm), the robot stops. With a servo sensor, the ultrasonic sensor rotates and scans left and right to determine the direction with more free space. The robot then turns toward the safer direction and moves forward before attempting to return to the line.

6.3. Obstacle Avoidance Algorithm Code

```
#include <Servo.h>

/* ==PIN DEFINITIONS== */
// Line Sensors
#define LFSensor_0 A0
#define LFSensor_1 A1
#define LFSensor_2 A2
#define LFSensor_3 A3
#define LFSensor_4 A4
// Motor Driver
#define speedPinR 3        // RIGHT PWM (ENA)
#define RightMotorDirPin1 12
#define RightMotorDirPin2 11

#define speedPinL 6        // LEFT PWM (ENB)
#define LeftMotorDirPin1 7
#define LeftMotorDirPin2 8
// Ultrasonic Sensor
#define Echo_PIN 2
#define Trig_PIN 10
// Servo
#define SERVO_PIN 9
#define FAST_SPEED 150  // speeds
#define MID_SPEED 140
#define SLOW_SPEED 130
// Obstacle threhold
const int distancelimit = 5;
Servo head;  // servo named head
```

Pin definitions

```
void setup(){
  //motor control
  pinMode(RightMotorDirPin1, OUTPUT);
  pinMode(RightMotorDirPin2, OUTPUT);
  pinMode(LeftMotorDirPin1, OUTPUT);
  pinMode(LeftMotorDirPin2, OUTPUT);
  pinMode(speedPinL, OUTPUT);
  pinMode(speedPinR, OUTPUT);
  //ultrasonic sensor
  pinMode(Trig_PIN, OUTPUT);
  pinMode(Echo_PIN, INPUT);
  //line sensors
  pinMode(LFSensor_0, INPUT);
  pinMode(LFSensor_1, INPUT);
  pinMode(LFSensor_2, INPUT);
  pinMode(LFSensor_3, INPUT);
  pinMode(LFSensor_4, INPUT);
  // initial motor state
  stop_Stop();
  // servo
  head.attach(SERVO_PIN);
  head.write(90);
  // serial
  Serial.begin(9600);
}

void loop(){
  auto_tracking();
}
```

Initialisation

```
void go_Forward(void){
  digitalWrite(RightMotorDirPin1, HIGH);
  digitalWrite(RightMotorDirPin2, LOW);
  digitalWrite(LeftMotorDirPin1, HIGH);
  digitalWrite(LeftMotorDirPin2, LOW);
}
```

Both motors spin forward →straight ahead.

```
void go_Left(int t=0){
  digitalWrite(RightMotorDirPin1, HIGH);
  digitalWrite(RightMotorDirPin2, LOW);
  digitalWrite(LeftMotorDirPin1, LOW);
  digitalWrite(LeftMotorDirPin2, HIGH);
  delay(t);
}
```

Right = forward, Left = backwards. →Turn left

```
void go_Right(int t=0){
  digitalWrite(RightMotorDirPin1, LOW);
  digitalWrite(RightMotorDirPin2, HIGH);
  digitalWrite(LeftMotorDirPin1, HIGH);
  digitalWrite(LeftMotorDirPin2, LOW);
  delay(t);
}
```

Right = backwards, Left = forward. →Turn right

```
void stop_Stop(){
  digitalWrite(RightMotorDirPin1, LOW);
  digitalWrite(RightMotorDirPin2, LOW);
  digitalWrite(LeftMotorDirPin1, LOW);
  digitalWrite(LeftMotorDirPin2, LOW);
}
void set_Motorspeed(int speedL, int speedR){
  analogWrite(speedPinL, speedL);
  analogWrite(speedPinR, speedR);
}
```

All LOWs → Stop

```
long watch() {
  digitalWrite(Trig_PIN, LOW);
  delayMicroseconds(5);
  digitalWrite(Trig_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(Trig_PIN, LOW);

  long duration = pulseIn(Echo_PIN, HIGH);
  long distance = duration * 0.034 / 2;  //cm

  return distance;
}
```

Measuring Distance. duration = the time the sound pulse bounces off an object and goes back. So the real distance needs to be halved.

```
int sensor[5];
// convert into 5-chracter binary
String read_sensor_values(){
  sensor[0] = !digitalRead(LFSensor_0);  // 1->0, 0->1
  sensor[1] = !digitalRead(LFSensor_1);
  sensor[2] = !digitalRead(LFSensor_2);
  sensor[3] = !digitalRead(LFSensor_3);
  sensor[4] = !digitalRead(LFSensor_4);

  int sensorvalue = 32 +
        sensor[0] * 16 +  //leftmost sensor = bit 4 (16)
        sensor[1] * 8 +   // bit 3 (8)
        sensor[2] * 4 +  // centre = bit 2 (4)
        sensor[3] * 2 +  // bit 1 (2)
        sensor[4];   // rightmost bit 0(1)

  String s = String(sensorvalue, BIN);
  s = s.substring(1, 6);  // if 6 characters: convert last 5, and add 32
  return s;
}
```

Converting 5 infrared line sensor readings into a 5-character binary string. Quickly detect the black line from the infrared line sensor.

```
bool obstacle_detected(){
  long d = watch();
  return d > 0 && d < distancelimit;
}
```

```
void avoid_obstacle(){
  stop_Stop();  // immediate stop
  delay(100);

  head.write(150); //servo turns 60 degree left
  delay(300);
  int leftDist = watch(); // diatance on left

  head.write(30); // 60 degree right
  delay(300);
  int rightDist = watch(); // distance on right

  head.write(90); //forward
  // move to more space direction
  if (leftDist > rightDist){
    go_Left();
  }else{
    go_Right();
  }
  set_Motorspeed(FAST_SPEED, FAST_SPEED);
  delay(400);
  stop_Stop();
  return_to_line();
}
```

(90-150)

Turns to a direction with more space.

Simulate avoidance, then find an alternative route to move.

```
void return_to_line(){
  Serial.println("Returning to line");
  // 3 servo attempts to fine line
  for (int att = 0; att <3; att ++){
    if (att%2 == 0){
      go_Left();
      Serial.println("Searching left");
    } else {
      go_Right();
      Serial.println("Searching right");
    }
    set_Motorspeed(MID_SPEED, MID_SPEED);

    for (int i = 0; i <10; i++){ // Check line sensor 10 times,
      delay(100);                read bit pattern
      String val = read_sensor_values();
      // found black line
      if (val !="00000"){
        Serial.println(val);
        go_Forward();
        set_Motorspeed(SLOW_SPEED, SLOW_SPEED);
        delay(100);
        return;
      }
    }
    stop_Stop(); // pause before changing direction
    delay(200);
  }
}
void auto_tracking(){
  String val = read_sensor_values();
  Serial.println(val);

  if (obstacle_detected()) {
    avoid_obstacle();
    return;
  }
  if (val == "00000"){
    return_to_line();
    return;
  }
  if (val == "10000"){ go_Left(); set_Motorspeed(FAST_SPEED, FAST_SPEED);}
  else if (val == "00001"){ go_Right(); set_Motorspeed(FAST_SPEED, FAST_SPEED);}
  else if (val == "01000" || val == "01100" || val == "00100"){ go_Forward(); set_Motorspeed(0, FAST_SPEED);}
  else if (val == "00010" || val == "00011" || val == "00110"){ go_Forward(); set_Motorspeed(FAST_SPEED, 0);}
  else{ go_Forward(); set_Motorspeed(MID_SPEED, MID_SPEED);}
}
```

Servo Left->right -> left

After obstacle avoidance, the robot should return to the line by continuously searching it in different directions. Any value except "00000" means the infrared sensor detected the line. The robot will move forward slowly, then back to auto_tracking().

Read 5 IR sensors and convert them into a 5-bit pattern. (1: black, 0: white).

First, check the ultrasonic sensor and if it is close to an obstacle, stop and run the avoid_obstacle() algorithm. Exit.

Then, check if the IR sensors return "00000", then run return_to_line() to return to the line track.

If there is no obstacle and the line is visible, then carry out line following logic – constantly adjust to ensure staying on the centre of the line and move forward.

6.4. Wall Following Algorithm Flowchart

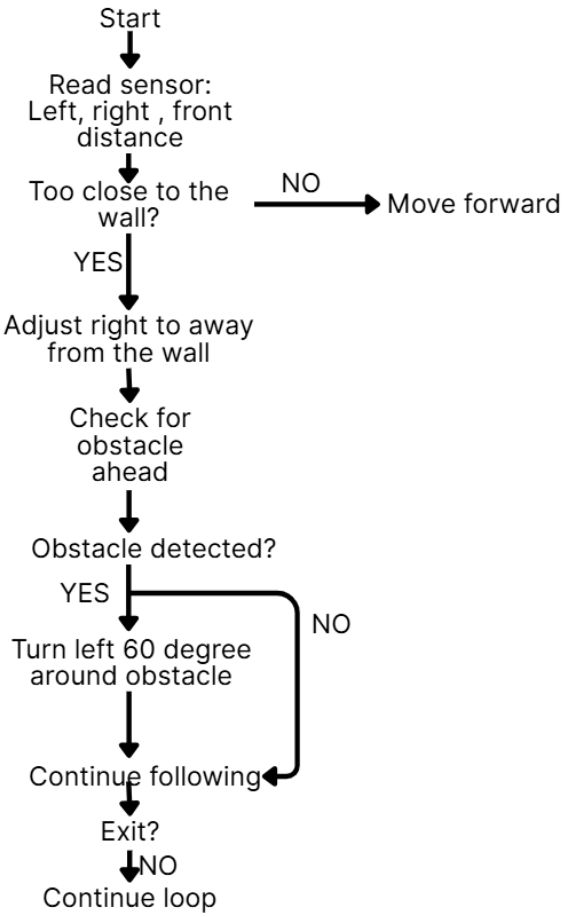


Figure 16

Figure 17

The wall-following algorithm uses an ultrasonic sensor to consistently scan left, right and front. It determined the distance to the left or right wall and decided where to move. For example, if it sensed too close to the left wall, it would adjust its position to steer right, and if it sensed too close to the right wall, it would steer to turn left. The distance was set to 15 cm in the laboratory. At the same time, obstacle avoidance in front is prioritised over wall following.

7. Testing the Ultrasonic Sensor

7.1. Methodology

The experiment aimed to measure distance using an ultrasonic sensor attached to a mobile robot and to evaluate the sensor's accuracy and reliability. The ultrasonic sensor was connected to the Arduino Uno microcontroller, with the trigger on digital pin D10 and the echo on digital pin D2. The ultrasonic code without the LCD part was uploaded and tested on a mobile robot, and the measurement outputs were recorded via the Arduino IDE serial monitor. For visualisation and simulation in Tinkercad, a 16*2 LCD was added to display real-time distance readings in centimetres and inches. This observed the sensor behaviour in a virtual environment. The results obtained from the physical robot experiment are presented in Section 5.1.2.

To evaluate the ultrasonic algorithm, the error between the actual distances and the readings displayed in the Arduino IDE serial monitor was calculated. A centimetre ruler was used to set seven different distances (10, 15, 20, 25, 30, 35, 40 cm). The robot was placed in a fixed position to detect the distance between a moving object in front of it. For each distance, three readings were collected from the serial monitor that were used to calculate the average measured distance. These averages were used to compare with the actual distances to determine the measurement error in cm. This allows quantitative evaluation of the ultrasonic sensor's accuracy in distance detection.

```
#include <LiquidCrystal.h>

// pins
#define Echo_PIN 2    // Ultrasonic Echo pin
#define Trig_PIN 10   // Ultrasonic Trig pin

// LCD pins
LiquidCrystal lcd(13, 12, 11, 9, 8, 7);

//setup
void setup() {
  // initialise serial communication
  pinMode(Trig_PIN, OUTPUT);
  pinMode(Echo_PIN, INPUT);

  Serial.begin(9600);
  // initialise LCD
  lcd.begin(16, 2);
  lcd.print("Ultrasonic Test");
  delay(1000);
  lcd.clear();
}
// loop repeat
void loop() {
  long duration, inches, cm;

  // Trigger ultrasonic sensor
  // Ping is triggered by a HIGH pulse of >2 ms
  // short LOW pulse beforehand to ensure a clean HIGH pulse.
  digitalWrite(Trig_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(Trig_PIN, HIGH);
  delayMicroseconds(5);
  digitalWrite(Trig_PIN, LOW);

  // echo time
  duration = pulseIn(Echo_PIN, HIGH);

  // convert the time into a distance
  inches = microsecondsToInches(duration);
  cm = microsecondsToCentimeters(duration);
```

```
cm = microsecondsToCentimeters(duration);

//serial Output
Serial.print(inches);
Serial.print(" in, ");
Serial.print(cm);
Serial.println(" cm");

//LCD Output
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Dist: ");
lcd.print(cm);
lcd.print(" cm");

lcd.setCursor(0, 1);
lcd.print("Dist: ");
lcd.print(inches);
lcd.print(" in");

delay(50);
}

// converting pulse duration to diistance using the speed of sound.
long microsecondsToInches(long microseconds) {
  return microseconds / 74.0 / 2;  // Ping: 73.746 ms per inch
}

long microsecondsToCentimeters(long microseconds) {
  return microseconds / 29 / 2;  // speed is around 29ms per cm
}
```

7.2. Results

Actual distance (cm)	Measured distance (cm)			Average (cm)	Error (cm)
	1	2	3		
10	10	11	10	10	0
15	16	16	15	16	1
20	20	20	21	20	0
25	25	26	27	26	1
30	30	30	31	30	0
35	35	36	34	35	0
40	40	41	39	40	0

Figure 18

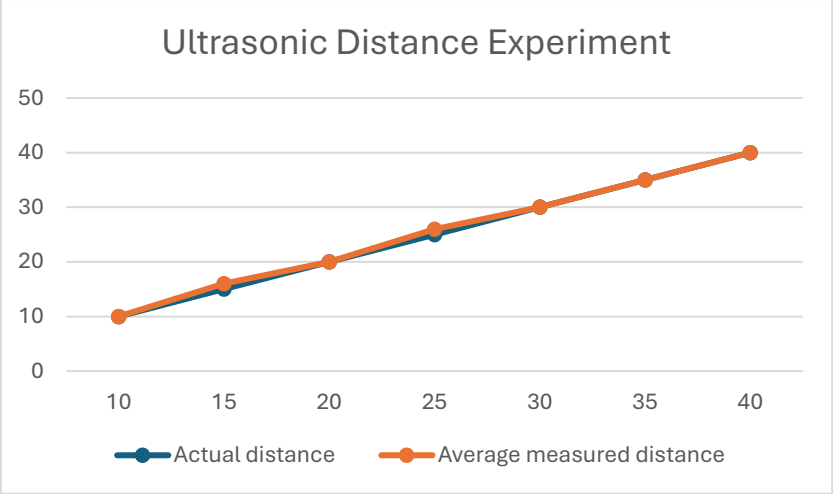


Figure 19

7.3. Discussion

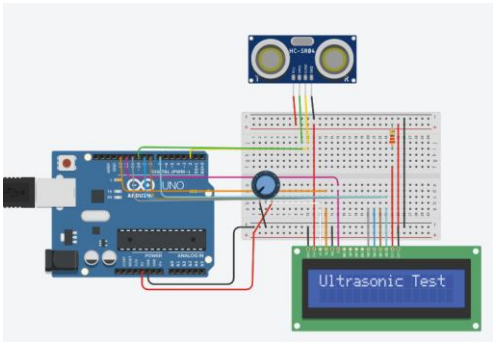


Figure 20

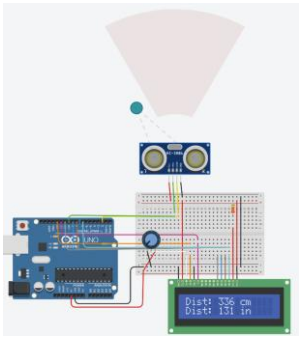


Figure 21

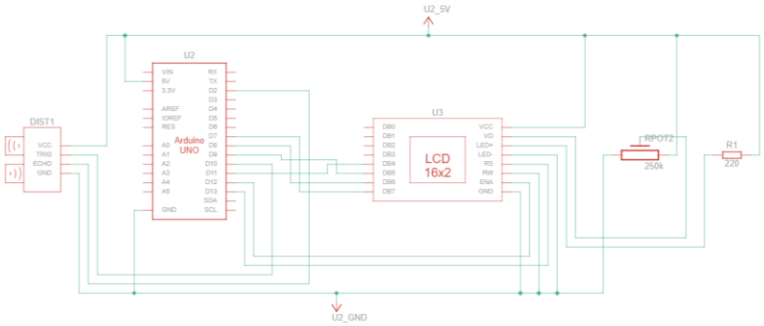


Figure 22

Figures 18 & 19 showed that the measured distances from the robot are very close to the actual distance, with some readings having minor +1/-1 errors. The average measured distance for testing closely matched the actual distance, with an error of 0 or 1 cm. These small errors may be caused by environmental factors such as sensor noise and sudden changes. Overall, the ultrasonic sensor performed with high accuracy and reliability within the short-range distance measurements(10-40cm) on the mobile robot.

Tinkercad simulation experiment (FIGURE23-25) showed a similar trend, with minor errors of ±1 around the actual distances. Analysing the floating distance readings, mobile robot experiments could be influenced by variations in echo timing and physical environmental factors such as sensor noise and temperature. In the Tinkercad experiment, the simulation approximates the distance based on the object's position and virtual environment noise.

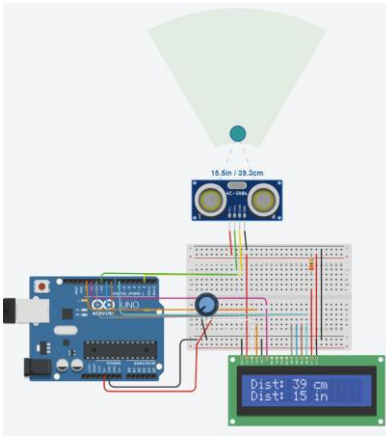


Figure 23

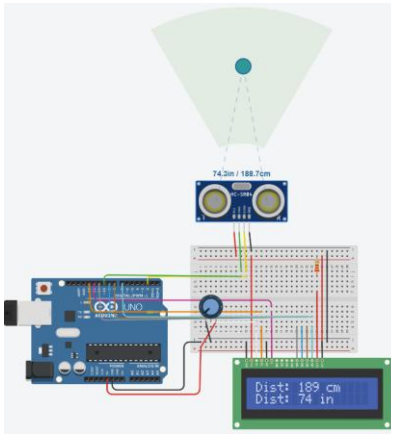


Figure 24

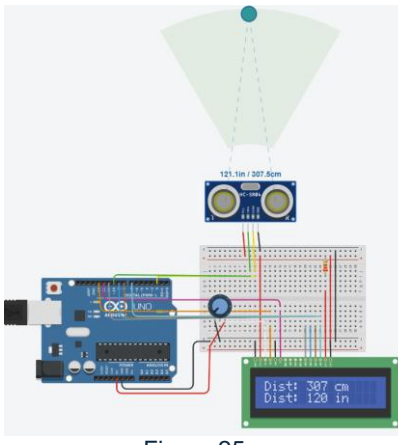


Figure 25

8. Conclusions and Future Work

The robot successfully followed a black line using a five-channel infrared tracking sensor, avoided obstacles and wall-following using an ultrasonic sensor mounted on a servo. and avoided obstacles. Experimental results showed that the ultrasonic sensor read accurate short-range distance measurements with a maximum observed error of ±1 cm. The line tracking algorithm with obstacle avoidance and return to line behaviour enabled the robot to navigate a cluttered environment autonomously. The wall following allowed the robot to interface with a space.

However, several limitations were observed. First, the navigation was sensitive to sensor noise because the control logic depended on the sound travel distance to navigate to the obstacle. Second, the robot struggled to turn with wide black lines, where the sensor pattern could not resolve a clear steering decision. As the loop runs forever, future work could include closed-loop control to reduce sensitivity to the sound. Third, path planning could be insufficient in very complex environments because the robot had no memory about obstacle positions and instead rescanned the surroundings upon each detection. In this case, a mapping system may be helpful to allow the robot store obstacle positions or create a 3D space rather than repeatedly scanning the environment. Finally, sometimes the robot locked onto the nearest black line instead of returning to the intended path after executing an avoidance algorithm. This may be resolved by implementing more advanced algorithms, such as Bug algorithms, which I have not done in the lab.

9. References

1. Pictures:

2. OSOYOO V2.1 Robot Car - https://osoyoo.com/2020/05/12/v2-1-robot-car-kit-for-arduino-tutorial-introduction/
3. UNO R3 Board - https://osoyoo.com/2020/05/12/v2-1-robot-car-kit-for-arduino-tutorial-introduction/
4. Arduino Software - https://www.arduino.cc/
5. Tinkercad Ultrasonic distance - https://www.tinkercad.com/things/9zBAAuCPpLQ-ultrasonic-speed-2/editel?sharecode=3ur0uSCTiw-iCgbdy9BRabynY1h4CW0lWW1-qDrFZ-A
6. Line-tracking Tinkercad - https://www.tinkercad.com/things/4J4cfL8tqBd/editel?sharecode=RZYpQyPLufSqfaIJ9ggbCVtS_6ivfRsxeya4yPvW7V0